

Name of the Student	U. Somesh Kumar
Reg. No	192425019
GitHub Link	https://github.com/someshkumar12345/Deep-Learning-MLA0403

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning
SLOT :

COURSE CODE: MLA04
Faculty : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Course Outcome Weightage: 35%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark : 25

Code of Conduct:

I U. Somesh Kumar (192425019 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

U. Somesh Kumar

Signature of the student:

U. Somesh Kumar
Name of the student:

Criteria	Subject Knowledge and Conceptual Understanding (2)	Technical Accuracy and Application of Concepts (3)	Problem-Solving and Analytical Thinking (2)	Communication Skills and Clarity of Explanation (2)	Confidence, Presentation and Professional Behaviour (1)	Total (10)
Weightage	20 %	30%	20%	20 %	10 %	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Faculty Signature :

Total Marks:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

Answer:

1. Problem Identification

A manufacturing company collects sensor data from machines to identify defective products. The sensor data may contain hundreds of features such as:

- Temperature
- Pressure
- Vibration
- Speed
- Voltage
- Current
- Humidity
- Motor condition

High-dimensional data is difficult to analyze directly. Therefore, an **Undercomplete Autoencoder** can be used to learn a compact representation of the sensor data.

Objective

The system should:

- Reduce the dimensionality of sensor data.
- Learn important features automatically.
- Remove unnecessary information.
- Reconstruct the original input.
- Provide useful features for defect detection.

2. Proposed Architecture

Sensor Input



Input Layer

100 Features



Dense Layer

64 neurons

ReLU



Dense Layer

32 neurons

ReLU



LATENT REPRESENTATION

8 neurons

Linear/ReLU



Dense Layer

32 neurons

ReLU



Dense Layer

64 neurons

ReLU

|



Output Layer

100 neurons

Linear

|



Reconstructed Sensor Data

3. Encoder

The encoder converts the high-dimensional input into a smaller representation.

Example:

$$100 \rightarrow 64 \rightarrow 32 \rightarrow 8$$

The **8-dimensional latent vector** contains the most important information about the sensor data.

Activation

ReLU is used in hidden layers:

$$ReLU(x) = \max(0, x)$$

It introduces non-linearity and helps the network learn complex relationships.

4. Latent Representation

The latent layer contains only **8 features**.

For example:

Original data:

[100 sensor features]



Latent vector:

[z1,z2,z3,z4,z5,z6,z7,z8]

The latent representation acts as a compressed version of the original sensor data.

5. Decoder

The decoder reconstructs the original sensor data.

$$8 \rightarrow 32 \rightarrow 64 \rightarrow 100$$

The final output should be as close as possible to the original input.

6. Reconstruction Loss

For continuous sensor data, **Mean Squared Error (MSE)** is suitable.

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Where:

- x_i = original sensor value
- $\hat{x}_i$ = reconstructed value

The model tries to minimize the reconstruction error.

7. Training Process

1. Collect sensor data.
2. Normalize the data.
3. Divide data into training and testing sets.

4. Give sensor data to the encoder.
5. Generate the latent representation.
6. Decode the latent representation.
7. Compare original and reconstructed data.
8. Calculate MSE loss.
9. Update weights using backpropagation.
10. Repeat for multiple epochs.

Optimizer

Adam optimizer can be used because it provides efficient gradient-based optimization.

8. Defect Detection

After training, the reconstruction error can be used to identify defective products.

Normal Product



Low Reconstruction Error



Normal

Defective Product



High Reconstruction Error



Potential Defect

A threshold can be selected:

$$Error > Threshold \Rightarrow Defective$$

9. Evaluation

Suitable measures include:

- Reconstruction MSE
- MAE
- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Expected Outcome

The model reduces 100 sensor features to only 8 meaningful features while retaining important information needed for defect detection.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

Answer:

1. Problem Identification

Healthcare organizations store patient information such as:

- Age
- Blood pressure
- Glucose
- Cholesterol
- Heart rate
- BMI
- Laboratory measurements

Real-world healthcare data may contain **noise, missing values and irrelevant variations**.

A regularized autoencoder can learn a robust representation while reducing overfitting.

2. Proposed Model

I propose a **Denoising Autoencoder with L2 regularization and Dropout**.

This is suitable because the input is intentionally corrupted with noise, and the network learns to reconstruct the clean patient representation.

3. Architecture

Noisy Patient Data

|



Input Layer

50 features

|



Dense

32 neurons

ReLU

|



Dropout

0.2

|



Dense

16 neurons

ReLU

|



LATENT SPACE

8 neurons



Dense

16 neurons

ReLU



Dense

32 neurons

ReLU



Output Layer

50 neurons

Linear



Clean Patient Representation

4. Regularization Mechanism

Dropout

A dropout rate of **0.2** can be applied.

During training, some neurons are randomly deactivated.

This prevents the model from depending too heavily on specific neurons.

L2 Regularization

An L2 penalty can be added to the loss:

$$L_{total} = L_{reconstruction} + \lambda \sum W^2$$

Where:

- $L_{reconstruction}$ = reconstruction loss
 - W = network weights
 - λ = regularization coefficient
-

5. Denoising Mechanism

Suppose the original data is:

[120, 80, 95, 180, 72]

Noise may produce:

[120, 84, 95, 0, 75]

The model receives the noisy data but tries to reconstruct:

[120, 80, 95, 180, 72]

Therefore, the model learns important patterns instead of memorizing noise.

6. Loss Function

For numerical healthcare features:

$$MSE = \frac{1}{n} \sum (x_i - \hat{x}_i)^2$$

Total objective:

$$L = MSE + \lambda \sum W^2$$

7. Training

1. Collect patient records.
2. Handle missing values.

3. Normalize numerical features.
 4. Add artificial noise.
 5. Feed noisy data to the encoder.
 6. Generate latent representation.
 7. Decode the representation.
 8. Compare reconstruction with clean data.
 9. Calculate reconstruction + regularization loss.
 10. Update parameters using Adam.
-

8. Advantages

- Reduces overfitting.
- Learns robust features.
- Handles noisy data better.
- Produces compact representations.
- Useful for downstream classification.

For example, the learned latent features can later be given to a classifier to predict disease risk.

9. Evaluation

Use:

- MSE
- MAE
- Classification accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Expected Outcome

The regularized autoencoder should produce a stable and useful representation of patient data while reducing the influence of noise and preventing excessive memorization.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

Answer:

Selected Model: Deep Belief Network (DBN)

Application

A recommendation system needs to learn complex user preferences and recommend products.

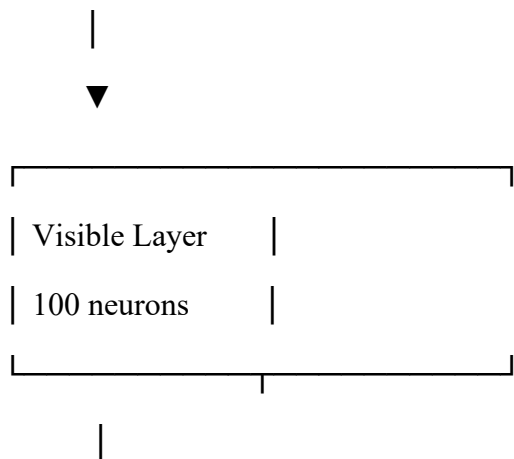
The system receives:

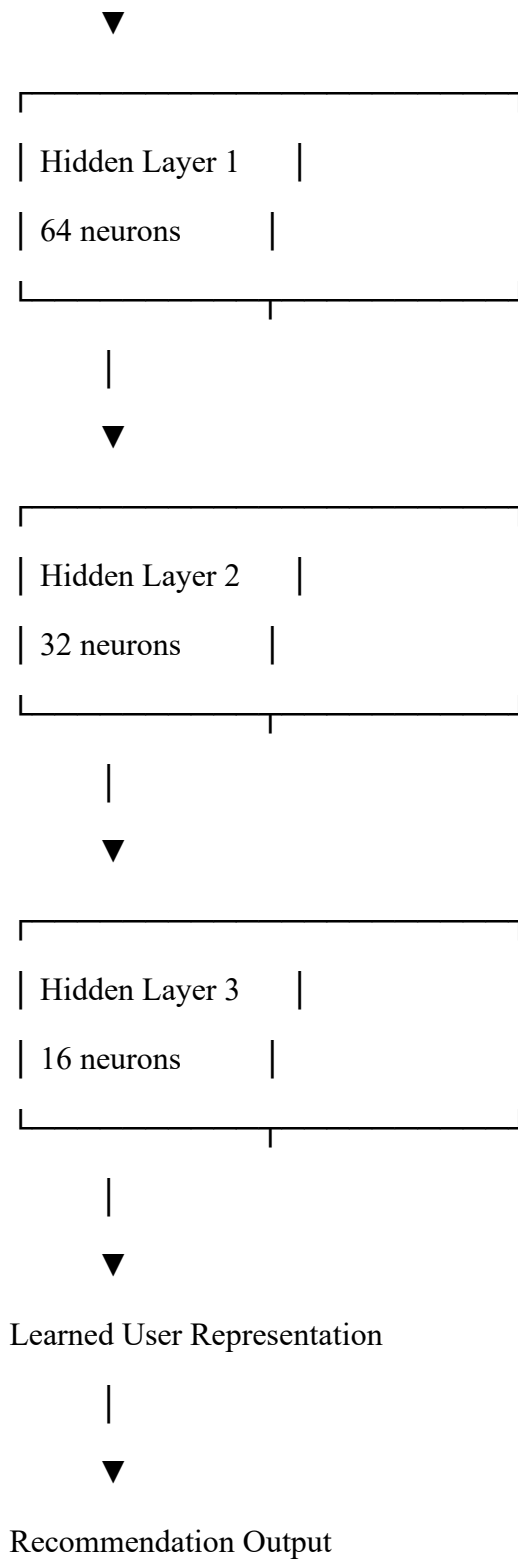
- User ID
- Product interactions
- Ratings
- Purchase history
- Browsing behaviour

A **Deep Belief Network** can learn hidden patterns in user behaviour.

1. Proposed Architecture

User-Item Interaction Data





2. Components

Visible Layer

The visible layer receives user-item interaction data.

Example:

Movie A = 1

Movie B = 0

Movie C = 1

Movie D = 1

Hidden Layers

Hidden layers learn latent relationships between users and products.

For example:

Action preference

Comedy preference

Price preference

Brand preference

3. Learning Process

A DBN is built using multiple layers of Restricted Boltzmann Machines (RBMs).

Each RBM consists of:

- Visible units
- Hidden units

There are connections between visible and hidden units, but not within the same layer.

4. RBM Architecture

Visible Layer

● ● ● ● ● ●

\\|\\|\\|/

\\|\\|/

● ● ● ●

Hidden Layer

The first RBM learns basic patterns.

Then the hidden representation becomes input to the next RBM.

RBM 1

Visible → Hidden 1

RBM 2

Hidden 1 → Hidden 2

RBM 3

Hidden 2 → Hidden 3

5. Learning Mechanism

DBN training consists of:

Phase 1 – Unsupervised Pre-training

Each RBM is trained individually.

The network learns useful representations without requiring labeled data.

Phase 2 – Fine-tuning

The complete network is optimized for the recommendation task.

6. Why DBN?

DBN is suitable because:

- It learns hidden representations.
 - It can work with large amounts of unlabeled data.
 - It captures complex relationships.
 - It can discover user preference patterns.
 - It can be used for recommendation and content generation.
-

7. Evaluation

For recommendation:

- Precision@K
- Recall@K
- F1-score
- NDCG
- RMSE
- MAE

Expected Outcome

The DBN learns hidden user preferences and generates personalized product recommendations.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

Answer:

Selected Model: Contractive Autoencoder

Application

Consider an **image-recognition system** used to identify objects from images.

Small changes in an image such as:

- Slight rotation
- Small translation
- Lighting variation
- Minor noise

should not completely change the learned representation.

Therefore, a **Contractive Autoencoder (CAE)** is suitable.

1. Architecture

Input Image

$128 \times 128 \times 3$



Convolution Layer

32 filters

3×3

ReLU



Max Pooling



Convolution Layer

64 filters

3×3

ReLU



LATENT REPRESENTATION

128 features



Dense / Decoder



Reconstructed Image

$128 \times 128 \times 3$

2. Contractive Mechanism

The main idea is to make the learned representation **insensitive to small changes in input**.

The normal reconstruction loss is:

$$L_{reconstruction} = ||x - \hat{x}||^2$$

A contractive penalty is added:

$$L_{contractive} = \left\| \frac{\partial h}{\partial x} \right\|_F^2$$

Therefore, the total loss becomes:

$$L = L_{reconstruction} + \lambda \left\| \frac{\partial h}{\partial x} \right\|_F^2$$

Where:

- x = input
 - $\hat{x}$ = reconstructed image
 - h = latent representation
 - λ = contractive regularization parameter
-

3. Robustness Mechanism

Suppose two images are almost identical:

Image A $\rightarrow$ Encoder $\rightarrow$ Representation A

Image B $\rightarrow$ Encoder $\rightarrow$ Representation B

Because the images differ only slightly:

Representation A $\approx$ Representation B

This provides stable features.

4. Training Process

1. Collect image dataset.
 2. Resize images.
 3. Normalize pixel values.
 4. Feed images into encoder.
 5. Generate latent representation.
 6. Reconstruct image using decoder.
 7. Calculate reconstruction loss.
 8. Calculate contractive penalty.
 9. Combine both losses.
 10. Update weights using Adam.
-

5. Advantages

- Robust against small input changes.
 - Learns stable representations.
 - Reduces sensitivity to noise.
 - Useful for image and signal processing.
 - Improves feature quality for downstream tasks.
-

6. Evaluation

Suitable measures:

- Reconstruction MSE
- MAE
- PSNR
- SSIM

- Classification accuracy
- Robustness under noise

Expected Outcome

The model should produce stable latent features even when the input image undergoes small variations.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

Answer:

Application

An organization has only a small number of training images for a product defect detection system.

For example:

Available defective images = 1,000

A GAN can generate additional realistic synthetic images.

1. Proposed GAN Architecture

A GAN consists of two networks:

Generator

Creates fake images.

Discriminator

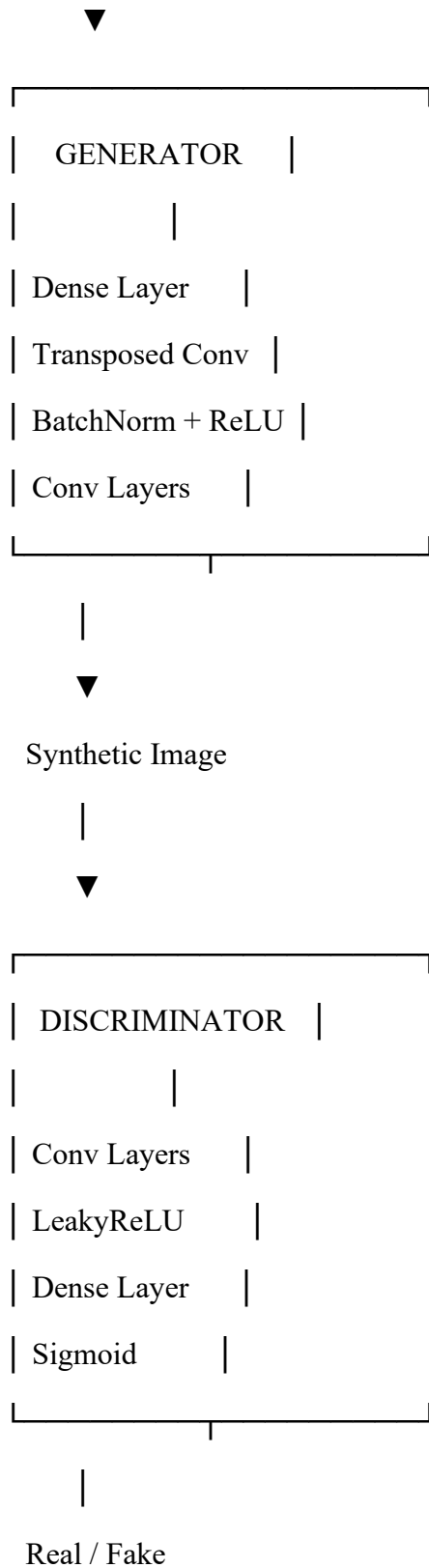
Determines whether an image is real or fake.

2. Architecture Diagram

Random Noise

$z \sim N(0,1)$

|



3. Generator

The generator receives random noise.

Example:

$$z \in R^{100}$$

It transforms this random vector into an image.

100-dimensional noise



Dense



Reshape



Transposed Convolution



Transposed Convolution



Generated Image

$128 \times 128 \times 3$

Activation Functions

Hidden layers:

ReLU

Output:

Tanh

Tanh is useful when image pixels are normalized to the range $[-1, 1]$.

4. Discriminator

The discriminator receives both real and generated images.

Image



Convolution

↓
LeakyReLU
↓
Convolution
↓
LeakyReLU
↓
Flatten
↓
Dense
↓
Sigmoid
↓
Real / Fake

The output is approximately:

$1 \rightarrow \text{Real}$

$0 \rightarrow \text{Fake}$

5. Adversarial Training

GAN training works like a competition.

Generator

Attempts to generate realistic images that fool the discriminator.

Discriminator

Attempts to correctly identify real and fake images.

The original GAN objective can be written as:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$$

6. Training Process

Step 1

Take a batch of real images.

Step 2

Generate random noise.

Step 3

Generator creates fake images.

Step 4

Discriminator receives:

Real image $\rightarrow$ Real

Fake image $\rightarrow$ Fake

Step 5

Calculate discriminator loss.

Step 6

Update discriminator.

Step 7

Train generator to fool discriminator.

Step 8

Repeat the process for many epochs.

7. Loss Functions

Discriminator Loss

The discriminator tries to maximize its ability to distinguish real and fake samples.

A binary cross-entropy formulation can be used.

Generator Loss

The generator tries to make generated samples classified as real by the discriminator.

A common non-saturating generator loss is:

$$L_G = -E[\log D(G(z))]$$

8. Evaluation

GAN quality can be evaluated using:

1. FID

Fréchet Inception Distance

Lower FID generally indicates that generated images are closer to real images in feature distribution.

2. Inception Score

Measures image quality and diversity.

3. Visual inspection

Generated images can also be evaluated by experts.

4. Downstream accuracy

Train the defect-detection classifier using real + synthetic images and compare its performance.

9. GAN vs Autoencoder

Feature	Autoencoder	GAN
Main purpose	Reconstruction/feature learning	Data generation
Architecture	Encoder + Decoder	Generator + Discriminator
Training	Reconstruction loss	Adversarial loss
Generated samples	Usually less realistic	Usually more realistic
Image quality	Moderate	High
Training stability	Easier	More difficult
Mode collapse	No	Possible
Suitable for synthetic images	Moderate	Excellent

10. Why GAN is Suitable

GAN is particularly suitable because the organization's main requirement is **generating realistic synthetic training images**.

The generator learns the distribution of real images and produces new samples.

For example:

Original Dataset



GAN



Synthetic Images



Combine



Real + Synthetic Dataset



Defect Classification Model

This can increase the amount and diversity of training data.

Overall Comparison of the Five Designs

Question Model		Main Application	Main Purpose
Q1	Undercomplete Autoencoder	Manufacturing	Dimensionality reduction
Q2	Regularized/Denoising Autoencoder	Healthcare	Robust feature learning
Q3	Deep Belief Network	Recommendation	Learning hidden distributions
Q4	Contractive Autoencoder	Image processing	Stable representation
Q5	GAN	Synthetic images	Data generation

Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.

- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.